JavaScript

Making Pages Do Things

CSCI 39548 — Practical Web Development

Section Goals

- Understand JavaScript as the programming language of the browser
- Work with variables, types, and operators
- Use functions, arrays, objects, and classes
- Manipulate the DOM and handle user events
- Make HTTP requests and persist data locally

Section Items

- Variables, Types, and Output
- Conditionals and Loops
- Strings and Numbers
- Functions
- Arrays and Array Methods
- Objects
- Scope, Closures, and Classes
- DOM Manipulation
- Events and Forms
- Async, Fetch, and Persistence

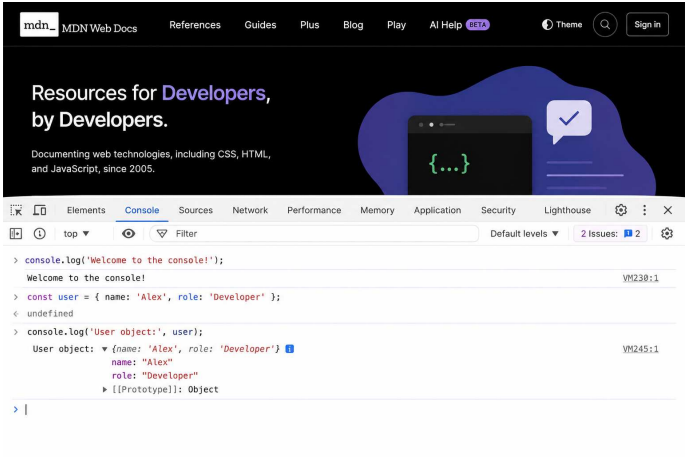
Section Items

- ▶ **Variables, Types, and Output**
- ○ Conditionals and Loops
- ○ Strings and Numbers
- ○ Functions
- ○ Arrays and Array Methods
- ○ Objects
- ○ Scope, Closures, and Classes
- ○ DOM Manipulation
- ○ Events and Forms
- ○ Async, Fetch, and Persistence

What JavaScript Is

- The programming language of the browser
- Runs in every browser, plus servers (Node.js), mobile, desktop
- HTML is the structure, CSS is the style, JS is the **behavior**
- The only language browsers understand natively

Where We're Going to Live



Three Ways JS Runs in a Browser

```
<!-- 1. Inline -->
<script>console.log("inline");</script>

<!-- 2. External file (preferred) -->
<script src="script.js"></script>

<!-- 3. Browser console (DevTools) -->
```

- Inline: quick experiments
- External: real projects (separation of concerns)
- Console: interactive playground

let vs const

```
const name = "Sourya";     // cannot reassign
let count = 0;             // can reassign
count = 1;                 // OK
name = "Bob";              // TypeError!
```

- `const` = default choice — value won't be reassigned
- `let` = for values that will change
- Forget `var` exists (we'll explain why later)

Primitive Types

```
const text    = "hello";      // string
const num     = 42;           // number
const flag    = true;         // boolean
const empty   = null;         // null (intentional nothing)
const missing = undefined;    // undefined (not yet set)
```

- Five primitive types you'll use daily
- BigInt and Symbol exist — skip them for now

typeof

```
typeof "hello"    // "string"
typeof 42         // "number"
typeof true       // "boolean"
typeof undefined  // "undefined"
typeof null       // "object"  <-- JS bug since 1995
```

- Returns a string telling you the type of a value
- The `null` one is a famous language bug — just memorize it

console.log

```
console.log("Hello");
console.log("Name:", name, "Age:", age);
console.log(`Name: ${name}, Age: ${age}`);
```

- Your best friend for inspecting what your code is doing
- Multiple arguments separated by commas
- Template literal teaser (covered fully in snapshot 03)

Operators

Category	Operators
Arithmetic	`+ - * / % **`
Comparison	`=== !== > < >= <=`
Assignment	`= += -= *= /=`
Logical	`&& || !`

Always Use ===

```
"5" == 5        // true  (coerces types!)
"5" === 5       // false (different types)

"0" == false    // true  (wat?)
"0" === false   // false (correct)
```

- == does silent type coercion — causes bugs
- === checks value AND type — always use this
- Same for !== over !=

Live Code: snapshot-01

- Open `snapshot-01/script.js` and walk through it
- Open DevTools → Console to see the output live

Section Items

- Variables, Types, and Output
- **Conditionals and Loops**
- Strings and Numbers
- Functions
- Arrays and Array Methods
- Objects
- Scope, Closures, and Classes
- DOM Manipulation
- Events and Forms
- Async, Fetch, and Persistence

Conditionals and Loops

Decisions and Repetition

if / else if / else

```
if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else {
    grade = "C";
}
```

- Basic decision-making
- Only one branch executes

Ternary Operator

```
const label = done ? "done" : "pending";

const message = age >= 18 ? "adult" : "minor";
```

- One-line if/else that returns a value
- `condition ? valueIfTrue : valueIfFalse`
- Use for short, single-value decisions — not complex logic

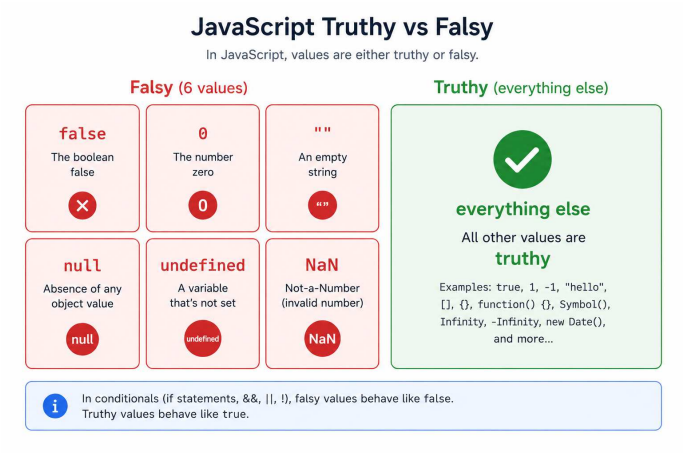
Truthy and Falsy Values

JS has only **6 falsy values** — everything else is truthy:

```
false, 0, "", null, undefined, NaN
```

- `"0"` is truthy (non-empty string)
- `[]` is truthy (empty array)
- `{}` is truthy (empty object)

The Falsy Six



Logical Operators and Short-Circuit

```
// Default value pattern
const displayName = name || "Guest";

// Guard pattern
const length = arr && arr.length;

// Negation
if (!done) { /* still pending */ }
```

- `&&` and `||` short-circuit and return **values**, not just booleans
- `||` returns the first truthy value (or the last)
- `&&` returns the first falsy value (or the last)

Classic for Loop

```
for (let i = 0; i < 5; i++) {
    console.log(i);  // 0, 1, 2, 3, 4
}
```

- Three parts: init, condition, update
- Use when you need the index

for...of

```
const colors = ["red", "green", "blue"];

for (const color of colors) {
    console.log(color);
}
```

- Iterates over array values directly
- Preferred when you don't need the index
- Cleaner than classic `for`

while

```
let attempts = 0;
while (attempts < 3) {
    console.log(`Attempt ${attempts + 1}`);
    attempts++;
}
```

Use when you don't know the iteration count up front.

break and continue

```
for (const item of items) {
    if (item === "stop") break;      // exit loop
    if (item === "skip") continue;  // skip to next
    console.log(item);
}
```

- `break` — exit the loop entirely
- `continue` — skip this iteration, move to next

Live Code: snapshot-02

- Walk through `snapshot-02/script.js`
- Watch the output in the console

Section Items

- Variables, Types, and Output
- Conditionals and Loops
- **Strings and Numbers**
- Functions
- Arrays and Array Methods
- Objects
- Scope, Closures, and Classes
- DOM Manipulation
- Events and Forms
- Async, Fetch, and Persistence

Strings and Numbers

Working with Text and Numbers

Template Literals

```
// Old way (concatenation)
const msg = "Hello, " + name + "! You are " + age + ".";

// New way (template literals)
const msg = `Hello, ${name}! You are ${age}.`;
```

- Backticks + ${expression}
- The modern way to build strings
- Any JS expression works inside ${}

Template Literals — Visual

String Methods

Method	What it does
`.length`	Character count (property, not method)
`.trim()`	Remove whitespace from both ends
`.toLowerCase()`	All lowercase
`.toUpperCase()`	All uppercase
`.includes(str)`	Contains substring?
`.startsWith(str)`	Starts with?
`.slice(start, end)`	Extract a portion
`.split(sep)`	Split into array
`.replace(a, b)`	Replace first match
`.replaceAll(a, b)`	Replace all matches

Strings Are Immutable

```
const greeting = "Hello";
greeting.toUpperCase();
console.log(greeting);  // "Hello" -- unchanged!

const upper = greeting.toUpperCase();
console.log(upper);     // "HELLO" -- new string
```

- Methods return NEW strings
- The original is never modified

String to Number Conversion

```
Number("42")           // 42
Number("hello")        // NaN

parseInt("42px")       // 42 (stops at non-digit)
parseFloat("3.14abc") // 3.14
```

- `Number()` — strict conversion
- `parseInt` / `parseFloat` — more forgiving, stops at first non-numeric character

Floating-Point Math

```
0.1 + 0.2              // 0.30000000000000004
0.1 + 0.2 === 0.3      // false!
```

- This is NOT a JavaScript bug
- Every language does this (IEEE 754 floating-point)
- For display: use `.toFixed(n)` — but it returns a **string**

The Math Object

Method	What it does
`Math.round(x)`	Nearest integer
`Math.floor(x)`	Round down
`Math.ceil(x)`	Round up
`Math.max(a,b,c)`	Largest value
`Math.min(a,b,c)`	Smallest value
`Math.random()`	Random 0–1 (exclusive)
`Math.PI`	3.14159...

NaN

```
const result = Number("hello");  // NaN
result === NaN                   // false! (NaN !== NaN)
Number.isNaN(result)             // true
```

- "Not a Number" — what you get from invalid math
- NaN is not equal to anything, including itself
- Use `Number.isNaN(x)` to check

The Pipe-String Hack

```
const raw = "1|Submit grading|false|45";
const parts = raw.split("|");
// ["1", "Submit grading", "false", "45"]
```

- Our todos are stored as pipe-delimited strings (for now)
- `.split("|")` breaks the string into an array
- This is intentionally ugly — we'll fix it in snapshot 06

Live Code: snapshot-03

- Walk through `snapshot-03/script.js`
- Pay attention to the pipe-delimited todo parsing at the end

Section Items

- Variables, Types, and Output
- Conditionals and Loops
- Strings and Numbers
- **Functions**
- Arrays and Array Methods
- Objects
- Scope, Closures, and Classes
- DOM Manipulation
- Events and Forms
- Async, Fetch, and Persistence

Functions

Reusable Blocks of Code

Three Ways to Write a Function

```
// Declaration
function add(a, b) { return a + b; }

// Expression
const add = function(a, b) { return a + b; };

// Arrow
const add = (a, b) => a + b;
```

- Declaration: classic, hoisted
- Expression: stored in a variable
- Arrow: modern, concise — our default going forward

Arrow Function Flavors

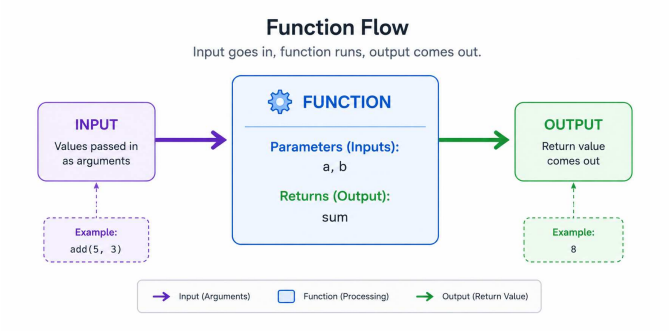
```
// Full body (multiple statements)
const greet = (name) => {
    const msg = `Hello, ${name}!`;
    return msg;
};

// Implicit return (single expression)
const double = (n) => n * 2;

// Single param (no parens needed)
const shout = name => name.toUpperCase();
```

Input → Function → Output

- Parameters go in, return value comes out
- Functions can also produce side effects (console.log, DOM changes)
- A function without `return` returns `undefined`



Default Parameters

```
function greet(name = "Guest") {
    return `Hello, ${name}!`;
}

greet("Alice")  // "Hello, Alice!"
greet()         // "Hello, Guest!"
```

- Provide a fallback value when no argument is passed
- Works with arrow functions too

Rest Parameters

```
function listAll(...items) {
    for (const item of items) {
        console.log(item);
    }
}

listAll("a", "b", "c");  // logs a, b, c
```

- `...items` collects any number of arguments into an array
- Must be the last parameter

Early Return

Nested (hard to read):

```
function getLabel(score) {
  if (score >= 90) {
    return "A";
  } else {
    if (score >= 80) {
      return "B";
    } else {
      return "C";
    }
  }
}
```

Early return (clean):

```
function getLabel(score) {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  return "C";
}
```

Bail out as soon as possible. Cleaner than nested if/else.

Functions Are Values

```
function runTwice(fn) {
    fn();
    fn();
}

runTwice(() => console.log("ping"));
// "ping"
// "ping"
```

- Functions can be passed as arguments
- Functions can be returned from other functions
- Foundation of `forEach`, `map`, event handlers

Live Code: snapshot-04

- Walk through `snapshot-04/script.js`
- The refactor: snapshot-03's todo printer is now split into `parseTodo`, `formatTodo`, `priorityLabel`, `totalRemainingMinutes`, `printTodoSummary`
- Same output, way cleaner code

Section Items

- Variables, Types, and Output
- Conditionals and Loops
- Strings and Numbers
- Functions
- **Arrays and Array Methods**
- Objects
- Scope, Closures, and Classes
- DOM Manipulation
- Events and Forms
- Async, Fetch, and Persistence

Arrays

Ordered Collections

Array Basics

```
const colors = ["red", "green", "blue"];
colors[0]                    // "red"
colors.length                // 3
colors[colors.length - 1]  // "blue" (last item)
```

- Ordered, zero-indexed
- `.length` is a property, not a method

Mutating Methods

Method	What it does
`push(item)`	Add to end
`pop()`	Remove from end
`unshift(item)`	Add to beginning
`shift()`	Remove from beginning
`splice(i, n)`	Remove n items at index i

These methods **change** the original array.

forEach

```
const names = ["Alice", "Bob", "Charlie"];

names.forEach((name) => {
    console.log(`Hello, ${name}!`);
});
```

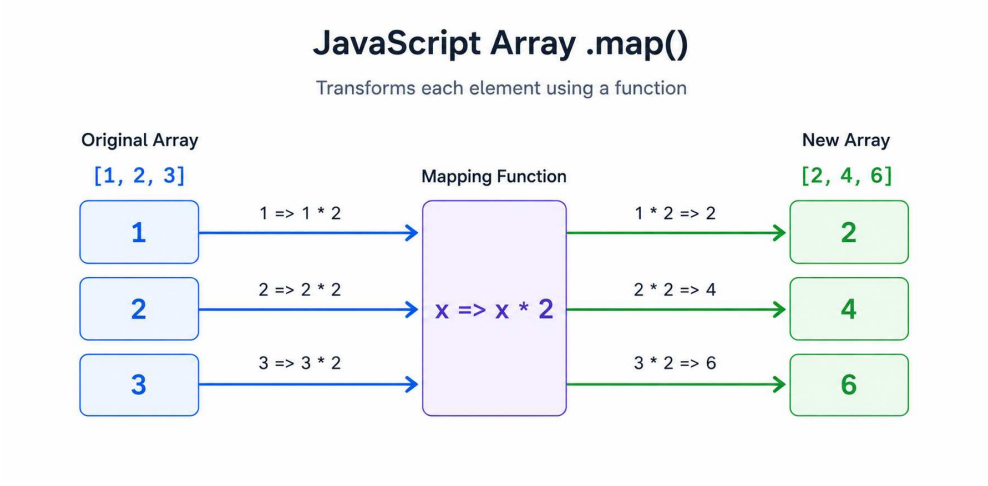
- Like `for...of` but with a function
- Returns nothing — use it for side effects only

map — The MVP

```
const nums = [1, 2, 3];
const doubled = nums.map((n) => n * 2);
// [2, 4, 6]
```

- Transforms every item, returns a NEW array
- Original array untouched
- You'll use this every single day

map Visualized

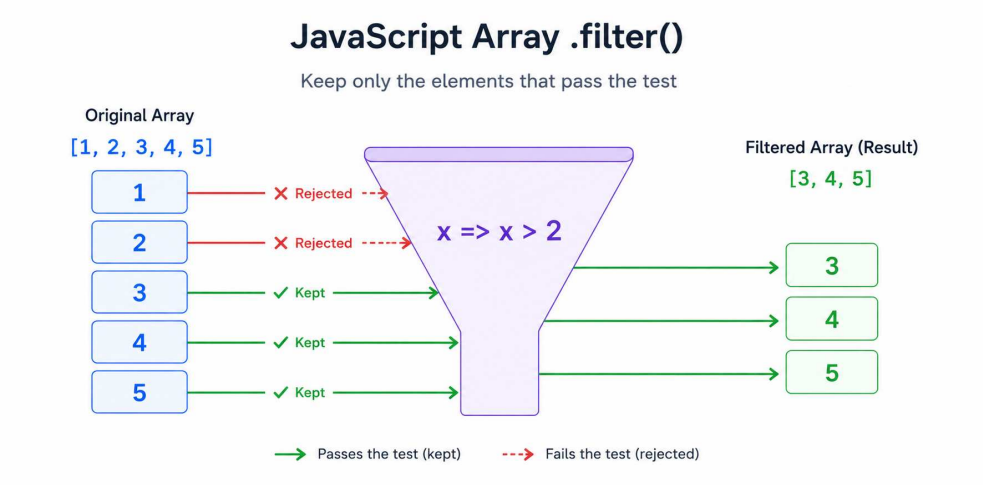


filter

```
const nums = [1, 2, 3, 4, 5];
const big = nums.filter((n) => n > 2);
// [3, 4, 5]
```

- Keep items that pass the test
- Returns a NEW array (original untouched)

filter Visualized



reduce

```
const nums = [1, 2, 3, 4];
const sum = nums.reduce((acc, n) => acc + n, 0);
// 10
```

- Collapse an array into a single value
- `acc` = accumulator (running total), `0` = initial value
- The mind-bending one — takes practice

find, findIndex, some, every

Method	Returns	What it does
`find(fn)`	item or `undefined`	First item passing the test
`findIndex(fn)`	index or `-1`	Index of first passing item
`some(fn)`	boolean	At least one passes?
`every(fn)`	boolean	ALL pass?

sort Gotcha

```
[10, 2, 33, 4].sort()
// [10, 2, 33, 4] -- sorted as STRINGS!

[10, 2, 33, 4].sort((a, b) => a - b)
// [2, 4, 10, 33] -- sorted as numbers
```

- `sort()` converts to strings by default
- Always pass a compare function for numbers
- WARNING: `sort` mutates the original array

Array Destructuring

```
const [first, second, ...rest] = [10, 20, 30, 40];
// first = 10, second = 20, rest = [30, 40]

const [, , third] = ["a", "b", "c"];
// third = "c" (skip with empty slots)
```

Spread ...

```
// Copy an array
const copy = [...original];

// Merge arrays
const all = [...arr1, ...arr2];

// Spread into function args
Math.max(...numbers);
```

- Unpack arrays into other arrays or function calls
- Creates a shallow copy (not a reference)

Chaining Array Methods

```
const result = todos
    .filter((t) => !t.done)
    .map((t) => t.text)
    .sort();
```

- Each method returns a new array → chain the next call
- Read top to bottom: filter → transform → sort

Live Code: snapshot-05

- Walk through `snapshot-05/script.js`
- The chained version of the todo printer
- Same output as snapshot-04, manual loops gone

Section Items

- Variables, Types, and Output
- Conditionals and Loops
- Strings and Numbers
- Functions
- Arrays and Array Methods
- **Objects**
- Scope, Closures, and Classes
- DOM Manipulation
- Events and Forms
- Async, Fetch, and Persistence

Objects

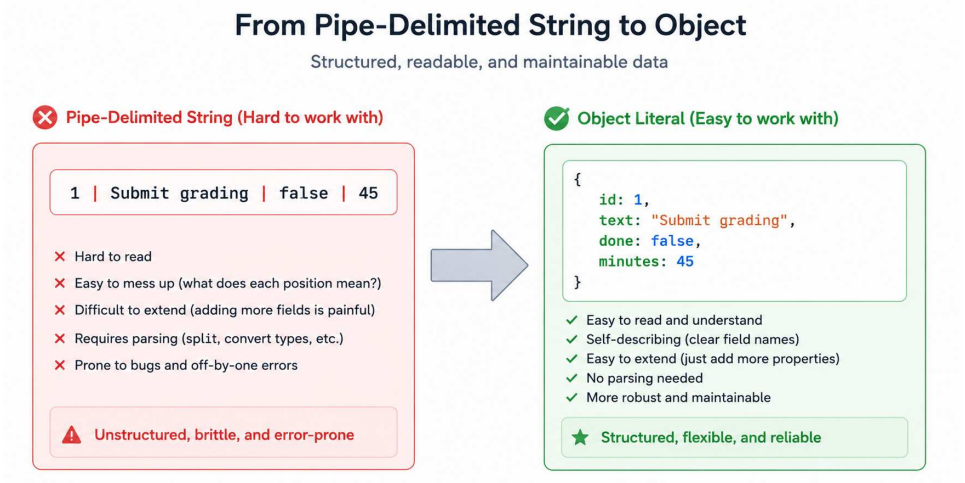
Bundling Related Data

The Hack We've Been Living With

```
const raw = "1|Submit grading|false|45";
// Parse by position... hope the order never changes
const parts = raw.split("|");
const id = Number(parts[0]);
const text = parts[1];
const done = parts[2] === "true";
```

Fragile, unreadable, error-prone. Time to kill this hack.

Pipe-Strings → Real Objects



Object Literal

```
const todo = {
    id: 1,
    text: "Submit grading",
    done: false,
    minutes: 45,
};
```

- { key: value } pairs
- Keys are strings (quotes optional if valid identifiers)
- Trailing comma is fine (recommended)

Dot vs Bracket Access

```
todo.text           // "Submit grading"
todo["text"]        // "Submit grading"

const field = "minutes";
todo[field]         // 45  (variable as key)
todo.field          // undefined (literal key "field")
```

- Dot: cleaner, use when you know the key
- Bracket: when the key is in a variable

Shorthand Syntax

```
const id = 1;
const text = "Submit grading";
const done = false;

// Old way
const todo = { id: id, text: text, done: done };

// Shorthand (same thing)
const todo = { id, text, done };
```

Object Destructuring

```
const { text, done, minutes } = todo;

// Rename
const { text: todoText } = todo;

// Default
const { priority = "medium" } = todo;

// Rest
const { id, ...rest } = todo;
```

Spread for Immutable Updates

```
const updated = { ...todo, done: true };
// { id: 1, text: "Submit grading",
//   done: true, minutes: 45 }
```

- Copy all properties, then override specific ones
- Original object unchanged
- This pattern is everywhere in React

Object.keys / values / entries

```
Object.keys(todo)      // ["id","text","done","minutes"]
Object.values(todo)    // [1, "Submit grading", false, 45]
Object.entries(todo)   // [["id",1], ["text","Sub..."], ...]

for (const [key, value] of Object.entries(todo)) {
    console.log(`${key}: ${value}`);
}
```

Computed Property Names

```
const field = "status";
const obj = { [field]: "active" };
// { status: "active" }
```

- `[expression]` as a key — evaluated at runtime
- Useful for dynamic keys

Methods

```
const counter = {
    count: 0,
    increment() {
        this.count++;
    },
    getCount() {
        return this.count;
    },
};

counter.increment();
counter.getCount();  // 1
```

Functions stored as properties. `this` refers to the object the method was called on.

Live Code: snapshot-06

- Walk through `snapshot-06/script.js`
- The dramatic switch: `parseTodo` now returns an object
- `formatTodo` destructures by NAME instead of position
- Show both versions side by side if screen allows

Section Items

- Variables, Types, and Output
- Conditionals and Loops
- Strings and Numbers
- Functions
- Arrays and Array Methods
- Objects
- **Scope, Closures, and Classes**
- DOM Manipulation
- Events and Forms
- Async, Fetch, and Persistence

Scope, Closures, Classes

Where Variables Live and How to Build Blueprints

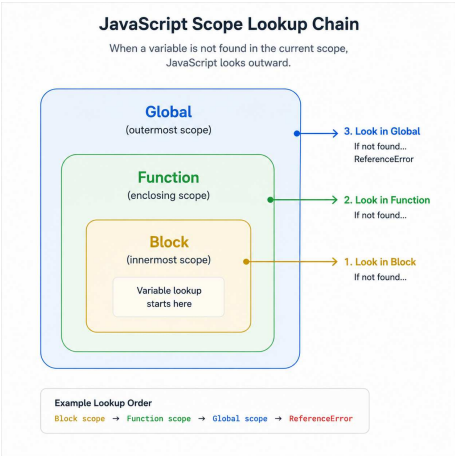
Block Scope

```
if (true) {
    const x = 10;
    let y = 20;
}
console.log(x);  // ReferenceError!
console.log(y);  // ReferenceError!
```

- `let` and `const` live inside { } where they're declared
- Outside the block, they don't exist

The Scope Chain

- JS looks for a variable in the current scope first
- If not found, checks the outer scope
- Keeps going until it reaches the global scope
- If still not found: `ReferenceError`



var and the Leak

```
if (true) {
    var leaked = "I escaped!";
}
console.log(leaked);  // "I escaped!" -- var ignores blocks
```

- `var` is function-scoped, not block-scoped
- It leaks out of `if`, `for`, `while` blocks
- Don't use `var`. Just recognize it in old tutorials.

Closures

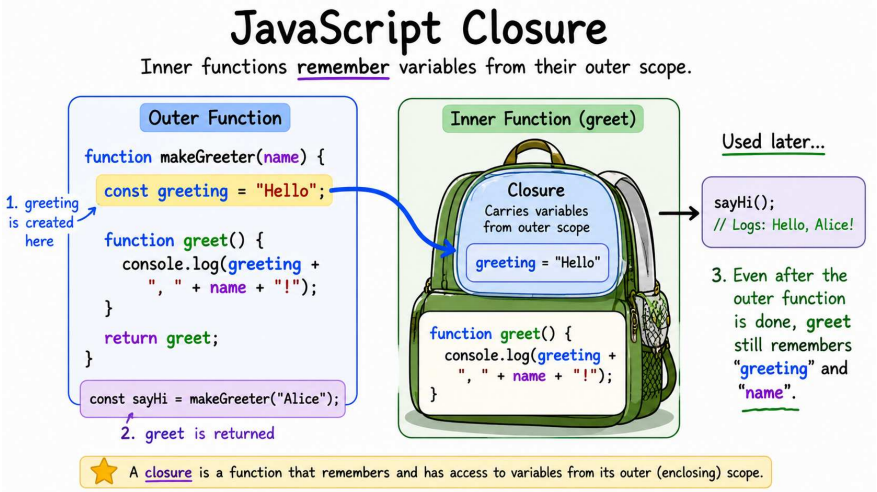
```
function makeGreeter(greeting) {
    return function(name) {
        return `${greeting}, ${name}!`;
    };
}

const hello = makeGreeter("Hello");
const hola = makeGreeter("Hola");

hello("Alice")  // "Hello, Alice!"
hola("Alice")   // "Hola, Alice!"
```

A function "remembers" variables from where it was created — even after the outer function returned.

Closures Visualized



Counter Factory

```
function makeCounter() {
    let count = 0;
    return {
        increment() { count++; },
        getCount()  { return count; },
    };
}

const a = makeCounter();
const b = makeCounter();
a.increment(); a.increment();
a.getCount()  // 2
b.getCount()  // 0 -- independent!
```

Each call creates a new, private `count`. Foundation of React hooks, event handlers, private state.

Classes — Anatomy

```
class Todo {
    constructor(id, text, minutes) {
        this.id = id;
        this.text = text;
        this.done = false;
        this.minutes = minutes;
    }

    toggle() {
        this.done = !this.done;
    }
}
```

- `class` = a blueprint for creating objects
- `constructor` runs when you call `new Todo(...)`
- `this` refers to the instance being created

Using a Class

```
const t = new Todo(1, "Submit grading", 45);
t.text      // "Submit grading"
t.done      // false
t.toggle();
t.done      // true
```

- `new` creates an instance
- Each instance has its own data, shared methods

The TodoList Class

```
class TodoList {
    constructor() {
        this.todos = [];
        this.nextId = 1;
    }
    add(text, minutes) { ... }
    remove(id) { ... }
    toggle(id) { ... }
    remainingMinutes() { ... }
    print() { ... }
}
```

Owns an array of todos. Exposes clean methods to manipulate them.

The Engine That Stays

- This `TodoList` class is the engine
- It stays **byte-for-byte unchanged** in snapshots 08, 09, and 10
- Only the surrounding UI code evolves:
  - Snapshot 08: DOM rendering
  - Snapshot 09: Event handling
  - Snapshot 10: Fetch + localStorage
- Clean separation of concerns

Live Code: snapshot-07

- Walk through `snapshot-07/script.js`
- Show the `Todo` and `TodoList` classes
- "This is the engine. From here on, we build UI on top without touching the class itself."